Ezra technical assessment for QA role by Maksim Demidov

Data To Be Used

1. Member Facing Portal: https://myezra-staging.ezra.com/
a. You can create members as you please without concern.
2. User Facing Portal: https://staging-hub.ezra.com/sign-in/
a. Username: michael.krakovsky+test_interview@functionhealth.com
b. Pwd: 12121212Aa
i. Please do not reset this password.
3. Stripe Test Data: https://docs.stripe.com/testing

Question 1

Part 1

The booking flow is integral to Ezra's business operation. Please go through the first three steps of the booking process including payment and devise 15 test cases throughout the entire process you think are the most important. When submitting the assignment, please return the test cases from the most important to the least important.

Ezra Booking Flow test cases

The 15 test cases below cover the first three steps of the Ezra member booking flow: (1) Scan Product Selection, (2) Location & Time Selection, and (3) Payment. They are ordered from most to least critical, prioritising revenue-impacting payment paths, core navigation correctness, data integrity, and then validation, security, and compatibility

.

Priority colour key: ■ 1–3 Critical ■ 4–10 High ■ 11–15 Medium

#	Test Case Title	Preconditions	Test Steps	Expected Result	Type
TC-001	**Successful payment with valid credit card**	User is logged in and has selected a scan and location	Enter valid card details (e.g., Visa 4242424242424242, future expiry 12/34, valid CVV) → Click 'Pay Now'	Payment processes successfully, confirmation screen is shown, confirmation email is sent, booking is created in the system	Happy Path
TC-002	**Payment declined with invalid/expired card**	User is logged in and on the payment step	Enter expired or invalid card details 4000000000000002 → Click 'Pay Now'	Payment is declined with a clear, user-friendly error message. No booking is created. User is prompted to re-enter card details	Negative

#	Test Case Title	Preconditions	Test Steps	Expected Result	Type
TC-003	**All scan products load and display correct pricing**	User is logged in and initiates the booking flow	Navigate to Step 1 of the booking flow → Review all scan products displayed	All scan types appear with correct names, descriptions, and prices	Functional
TC-004	**User can select an available imaging location**	User has selected a scan product in Step 1	Enter a state, or click Find closest centers to me → Select an available imaging facility from results	Relevant imaging centers are returned, displaying name, address, and available time slots. User can proceed after selection	Functional
TC-005	**Payment amount matches selected scan price exactly**	User has completed Steps 1 and 2 and is on the payment screen	Review order summary on payment page before submitting	Total displayed on payment screen matches the price of the selected scan. No unexpected fees or discrepancies	Data Integrity
TC-006	**HSA/FSA card accepted as valid payment method**	User is on the payment step	Enter a valid HSA or FSA card number → Click 'Pay Now'	HSA/FSA payment processes successfully. Booking is confirmed. Ezra's website states HSA/FSA may be eligible	Functional
TC-007	**User can select a scan and proceed to Step 2**	User is on Step 1 of the booking flow	Click on a scan product (e.g., Full Body MRI) → Click 'Continue' or 'Next'	Selected scan is highlighted/confirmed, and user is advanced to Step 2 (location/time selection)	Navigation
TC-008	**Integration with payment providers**	User is logged in and on the payment step	Select payment methods other than card (Affirm, Bank, Google Play)	User is redirected to a corresponding third-party widget/page to securely complete the next steps. The scan price is transferred correctly.	Integration
TC-009	**Promo/coupon code applies correct discount**	User has a valid promo code and is on the payment step	Enter a valid promo code in the coupon field → Apply	Discount is reflected correctly in the order total. Updated price is clearly shown before payment is submitted	Functional
TC-010	**Invalid promo code shows appropriate error**	User is on the payment step	Enter an expired or non-existent promo code → Apply	Error message is displayed stating the code is invalid or expired. Price is unchanged	Negative
TC-011	**Available time slots display correctly and can be selected**	User has selected an imaging facility	View the calendar/time slot picker → Select an available slot	Only available (non-booked) slots are shown as selectable. Past dates are not selectable. Selected slot is confirmed	Functional
TC-012	**Payment form validates required fields before submission**	User is on the payment step	Leave one or more required fields blank (e.g., card number, expiry, CVV, billing zip) → Click 'Pay Now'	Form submission is blocked. Validation errors highlight the missing/invalid fields with clear messages	Validation

#	Test Case Title	Preconditions	Test Steps	Expected Result	Type
TC-013	**User can navigate back to a previous step without losing selections**	User is on Step 2 or Step 3 (payment)	Click 'Back' button from Step 2 or payment page	User is returned to the previous step. Prior selections (e.g., scan type, location) are preserved and do not need to be re-entered	Navigation
TC-014	**Payment page is secured via HTTPS and sensitive fields are masked**	User has already paid with a credit card (saved in link), and is on the payment step again	Inspect the payment page URL and observe card number/CVV input fields	Page loads over HTTPS (padlock visible). Card number is masked (e.g., shown as Visa •••• 4242) and CVV field does not display entered characters in plain text	Security
TC-015	**Booking flow is functional on mobile browsers**	User accesses the member portal on a mobile device (iOS Safari / Android Chrome)	Complete Steps 1–3 (scan selection, location/time, payment) on a mobile viewport	All UI elements are correctly displayed and usable on mobile screen sizes. No elements overflow, overlap, or become inaccessible. Payment can be completed successfully	Compatibility

Part 2

For the top 3 test cases from part 1, please provide a description explaining why they are indicated as your most important.

TC-001 to TC-003 are ranked first because a broken payment path or missing product catalogue directly stops revenue generation.

Question 2

Part 1

Being privacy focused is integral to our culture and business model. Please devise an integration test case that prevents members from accessing other's medical data.
Hint: Begin Medical Questionnaire.

Cross-Member Medical Data Access Prevention test case: Prevent Member B from accessing or modifying Member A's Medical Questionnaire data

When a member begins the Medical Questionnaire, the system assigns it a unique identifier (encounterId) that is embedded in the URL or transmitted via API request parameters. If the back-end does not enforce ownership checks on every request, an authenticated but unauthorised member (Member B) may be able to read, submit, or manipulate another member's (Member A's) questionnaire simply by substituting that resource ID (mqSubmissions.id) — a vulnerability class known as Insecure Direct Object Reference (IDOR).

Medical questionnaires contain Protected Health Information (PHI): medical history, implant details, claustrophobia indicators, medications, and more. Unauthorised access constitutes a HIPAA violation and fundamentally breaks Ezra's privacy-first promise to members.

Test Actors

Actor	Account	Role in Test	Auth State
Member A	maxim.demidov+00@gmail.com (registered & verified)	Legitimate owner of the Medical Questionnaire. Initiates booking, begins questionnaire, captures resource ID from URL/network traffic.	Logged in — valid session token
Member B	maxim.demidov+01@gmail.com (separate registered account)	Malicious or curious member attempting to access Member A's questionnaire data using Member A's resource ID.	Logged in — separate valid session token

Preconditions

#	Condition
1	Two distinct member accounts exist in the staging environment (Member A and Member B), each fully verified with individual email addresses.
2	Both accounts are able to reach the booking flow and the 'Begin Medical Questionnaire' step.
3	Member A's account has an active (in-progress or submitted) Medical Questionnaire with a known resource ID (`/api/medicaldata/forms/mq/submissions/3565/data`).
4	Member B is authenticated in a separate browser session (different browser, incognito window, or separate device) and does NOT have any questionnaire of their own in progress at the time of the attack step.
5	Network interception tooling (e.g., browser DevTools) is available to capture the exact API request URLs and tokens used during Member A's session.

Integration Test Steps

PHASE 1 — Member A establishes the Medical Questionnaire resource

#	Actor	Action	Expected System Behaviour
1	**Member A**	Log in to the staging member portal with Member A's credentials. Navigate to the booking flow and select any scan product.	Successful authentication. Member A lands on the booking flow — scan selection step.
2	**Member A**	Complete scan selection (Step 1) and location/time selection (Step 2). Advance to the Medical Questionnaire step.	System creates a Medical Questionnaire resource tied exclusively to Member A's account. The questionnaire URL or API endpoint exposes a unique resource ID (encounterId).
3	**Member A**	Using browser DevTools (Network tab) or a proxy, record: (a) the full questionnaire URL including any resource ID path segments, and (b) Member A's session/auth token from request headers.	Resource ID and Member A's session token are captured for use in Phase 2. Member A does NOT complete or submit the questionnaire.

PHASE 2 — Member B attempts IDOR access using Member A's resource ID

#	Actor	Action	Expected System Behaviour
4	Member B	In a completely separate browser session, log in with Member B's credentials. Do NOT begin any questionnaire.	Member B is authenticated with their own session token.
5	Member B	[Attack 1 — Direct URL navigation] In Member B's browser, manually navigate to the exact questionnaire URL captured in Step 3 (containing Member A's resource ID).	System returns HTTP 403 Forbidden or HTTP 404 Not Found. Member B is NOT shown any of Member A's questionnaire data. Member B is logged out immediately.
6	Member B	[Attack 2 — API request with own token] Using DevTools, craft a direct API GET request to the questionnaire endpoint (e.g., GET `/api/medicaldata/forms/mq/submissions/3565/data`) using Member B's own valid session token in the Authorization header.	API returns HTTP 403 Forbidden. Response body does NOT contain any of Member A's data. E.g. `{ "message": "Authentication failed", "path": "/api/medicaldata/forms/mq/submissions/3565/data" }`
8	Member B	[Attack 3 — Mutation attempt] Craft an API POST request to Member A's questionnaire endpoint using Member B's own valid session token, attempting to overwrite questionnaire answers. E.g. POST `/api/medicaldata/forms/mq/submissions/3565/data` with payload: `{ "key": "address", "value": "123 Main St, New York, NY", "hasAnswer": true }`	API returns HTTP 403 Forbidden. No changes are persisted to Member A's questionnaire. Member A's data remains intact. `{ "message": "Authentication failed", "path": "/api/medicaldata/forms/mq/submissions/3565/data" }`

Part 2

Please devise HTTP requests from Part 1 to implement your test case. Submitting written HTTP requisitions is fine, you do not need to submit a postman project.

Log in as Member B to get an access token:

```
POST /individuals/member/connect/token HTTP/1.1
Host: stage-api.ezra.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 187

grant_type=password&scope=openid%20offline_access%20profile%20roles%20email&username=<MEMBER_EMAIL>&password=<MEMBER_PASSWORD>&client_id=F59A84B4-6E6B-4678-97A0-11C0F6E0719F
```

Save `access_token` from the response body. It will be used as Bearer token to auth in further requests.

Given Member A's encounterId (f5a738eb-ac55-47cd-86ae-c1c18c12e91b), get their submission id:

```
GET /diagnostics/api/medicaldata/forms/mq/submissions/f5a738eb-ac55-47cd-86ae-c1c18c12e91b/detail HTTP/1.1
Host: stage-api.ezra.com
Authorization: ••••••
```

Expected response:

```
200 OK

{

    "mqSubmissions": [

        {

            "id": 3565,

            "medicalFormKey": "mq",

            "memberId": "e917b5a1-6241-4c25-8e20-dd5de406b99d",

            "startedDate": "2026-04-03T22:46:22.8378011Z",

            "completedDate": "2026-04-04T20:50:21.9477141Z",

            "created": "2026-04-03T22:46:22.8378012Z",

            "encounterId": "f5a738eb-ac55-47cd-86ae-c1c18c12e91b"

        }

    ],

    "isFirstMQ": true

}
```

Attempt to access Member A's questionnaire data by Member B:

```
GET /diagnostics/api/medicaldata/forms/mq/submissions/3565/data HTTP/1.1
Host: stage-api.ezra.com
Authorization: ••••••
```

Expected response:

```
403 Forbidden

{

    "message": "Authentication failed",

    "path": "/api/medicaldata/forms/mq/submissions/3565/data"

}
```

Attempt to modify Member A's questionnaire data by Member B:

```
POST /diagnostics/api/medicaldata/forms/mq/submissions/3565/data HTTP/1.1
Host: stage-api.ezra.com
Content-Type: application/json
Authorization: ••••••

{

        "key": "address",

        "value": "123 Main St, New York, NY",

        "hasAnswer": true

}
```

Expected response:

```
403 Forbidden

{

    "message": "Authentication failed",

    "path": "/api/medicaldata/forms/mq/submissions/3565/data"

}
```

Part 3

At Ezra, we have over 100 endpoints that transfer sensitive data. What is your thought process around managing the security quality of these endpoints? What are the tradeoffs and potential risks of your solution?

How I Frame the Problem

One hundred endpoints cannot be meaningfully secured by manual testing alone. The math is straightforward: if each endpoint has even five security-relevant test cases, that is 500 checks that need to run every time the codebase changes. Any strategy that relies purely on a human running those checks will eventually produce gaps — either through release pressure, staff turnover, or simple oversight.

The question I therefore ask first is not 'how do I test these endpoints?' but rather: 'at what layer of the development lifecycle does each class of vulnerability get caught most cheaply and most reliably?' The answer to that drives the entire strategy.

The goal is not to be the last line of defence. It is to build a system where PHI-exposing bugs cannot reach production undetected — regardless of who wrote the code.

The Tiered Defence Strategy

I think about security coverage in the following tiers, each operating at a different speed and cost. They are complementary — no single tier catches everything, and the combination is what makes the system robust.

Tier	Layer	Mechanism	What it catches	Speed / trigger
1	**Shift-left (code)**	Peer review checklist	Missing auth guards, obvious injection sinks, hardcoded secrets	Every PR — seconds to minutes
2	**Contract tests**	Automated schema + auth assertions per endpoint in CI	Broken auth, missing 401/403, unexpected PHI in error bodies, schema drift	Every CI run — minutes
3	**Targeted integration suite**	One canonical test templated across all resource types that carry PHI	Cross-member data access, ownership enforcement gaps, write-path auth bypass	Nightly or per-deploy — minutes to ~1 hour
4	**Manual + pen-test**	Quarterly internal red-team of the highest-risk endpoint clusters; annual third-party pen-test	Chained multi-step attacks, novel logic flaws, social engineering vectors, findings missed by automation	Quarterly / annually — days

Endpoint Classification

The first practical task when dealing with 100+ endpoints is to classify them by risk, because testing effort is not free and it must be proportionate.

1. **Critical**. Directly reads, writes, or returns Protected Health Information (questionnaire, scan reports, member health data, physician notes, etc).
2. **High**. Authentication, authorisation, token issuance, account management (login, session refresh, password reset, member account update).

3. **Medium**. Financial data, booking state, scan scheduling (Booking creation, payment processing, appointment management, promo codes).
4. **Low**. Largely static, non-personal, low blast radius if exposed (Facility listings, scan product catalogue, pricing tiers, etc.).

Critical and High class endpoints get all tiers applied. Medium gets Tiers 1–3 on major releases. Low gets Tiers 1–2 only. This ensures the deepest scrutiny lands exactly where a breach would hurt most.

Tradeoffs

1. Speed vs. coverage. A comprehensive security suite adds minutes to CI. Keep Tier 1 & 2 under 5 minutes total by running them in parallel.
2. Automation vs. false confidence. Automated tests catch what they are written to catch. A novel business-logic flaw — e.g. a doctor account that can read any patient in its assigned facility — may not match any existing rule. Treat automation as floor coverage, not ceiling. Mandate a manual exploratory session on every new feature area before it ships. Document findings as new automated cases.

Tradeoff	The tension	How I would manage it
Speed vs. coverage	A comprehensive security suite adds minutes to CI.	Keep Tier 1 & 2 under 5 minutes total by running them in parallel.
Automation vs. false confidence	Automated tests catch what they are written to catch. A novel business-logic flaw — e.g. a doctor account that can read any patient in its assigned facility — may not match any existing rule.	Treat automation as floor coverage, not ceiling. Mandate a manual exploratory session on every new feature area before it ships. Document findings as new automated cases.
Maintenance burden	A large automated security suite that no one maintains becomes a source of noise — flaky tests, outdated schemas, suppressed alerts — which is worse than no suite because it destroys trust in the signal.	Assign explicit ownership. Each endpoint cluster has a team owner responsible for keeping its security tests green. QA owns the Insecure Direct Object References (IDOR) template; engineering owns the schema contracts.
Velocity impact on engineers	Blocking PRs on security findings will frustrate engineers if findings are noisy or the fix path is unclear.	Every automated finding must include a remediation link. Treat false positives with the same urgency as false negatives — a tool that cries wolf gets disabled.

Automation

Use Playwright to automate 2–3 of the test cases you created in Part 1. Please explain why you chose those test cases to automate. Please include:
● Trade-offs and assumptions.
● Include a short README.md with setup steps and your explanation notes.
● Comments or a README.md explaining trade-offs, assumptions, scalability, and what you would implement in the future.
● Submit a GitHub repo link containing all tests and documentation.
● Structure your automation scripts using a scalable model (preferably Page Object Model). Your submission must demonstrate architecture, coding, and design decisions required for production-level.

GitHub repository

https://github.com/max-demidov/ezra

This repo has a comprehensive README.md that includes installation and running instructions, project structure, trade-offs, assumptions, scalability, and future steps.

Selection explanation

I selected these two test cases because together they represent the two highest-risk failure modes in Ezra's product and demonstrate complementary automation approaches.

TC-001 — the booking happy path is the most business-critical flow in the system. If a member cannot complete registration, select a scan, and pay, Ezra generates no revenue. Automating this as a serial end-to-end suite gives the team a fast signal on every deploy that the full transaction path is intact — from a new member account through Stripe to a confirmed booking visible in the back-end. It also gave me the opportunity to demonstrate the Page Object Model across multiple pages, an API client for back-end verification, and how to handle third-party complexity like the Stripe iframe and a SPA with eventual consistency.

The IDOR test addresses Ezra's most severe privacy risk. Ezra handles Protected Health Information — medical questionnaire answers, scan results, medication history. A broken authorisation check on any PHI endpoint is not just a bug, it's a HIPAA violation and a direct breach of the trust that Ezra's entire brand is built on. This test goes beyond what a manual tester would typically check: it requires two independent authenticated sessions running simultaneously, purposefully crafting requests that a real user would never send, and asserting on HTTP status codes rather than UI state. That kind of adversarial, multi-actor scenario is exactly where automation adds value that manual testing cannot reliably replicate at scale.

The combination deliberately covers both layers of the stack — UI and API — and both directions of failure — something that should work and something that should be blocked. That felt like the most complete demonstration of a QA automation strategy within the scope of this assessment.